

Exploring the Role of Large Language Models as Artificial Tutors

Benedikt Zönnchen*, Martin Hobelsberger†, Gudrun Socher*
Veronika Thurner† and Sarah Ottinger*

*Munich Center for Digital Sciences and AI

†Department of Computer Science and Mathematics

*†Hochschule München University of Applied Sciences, Munich, Germany

zoennchen.benedikt@hm.edu, martin.hobelsberger@hm.edu, gudrun.socher@hm.edu,
veronika.thurner@hm.edu, sarah.ottinger@hm.edu

Abstract—As large language models (LLMs) become increasingly integrated into learning environments, their potential to enhance or hinder the acquisition of computational skills remains debated. This study investigates the role of generative AI (GenAI) tools, particularly Harvard’s CS50 Duck, in supporting programming education. Through a mixed-methods approach, we examine students’ perceptions, engagement, and practical application of the CS50 Duck within our Computational Thinking course. Our results indicate that while proficient students use GenAI tools to reinforce problem-solving skills, struggling students may over-rely on them, potentially bypassing critical learning processes. Survey and assignment data suggest that students value the non-judgmental feedback provided by the CS50 Duck, yet express nuanced views on GenAI’s role in formal assessments and programming education. We show that analysing chat histories can serve as a qualitative framework for examining the interactions between students and artificial tutors, while simultaneously offering critical insights into students’ learning processes and the challenges they encounter. This study also underscores the need for instructional strategies that guide responsible GenAI use and highlights the importance of educator involvement in integrating these tools effectively.

Index Terms—artificial intelligence, generative artificial intelligence, large language models, computing education, programming courses, artificial tutor

I. INTRODUCTION

In recent years, computer science education has undergone substantial digital disruptions, further intensified by the COVID-19 pandemic. Large language models (LLMs) have emerged as a promising tool in computing education, especially in addressing challenges students face when solving programming problems [1]. These models offer scalable and effective support in programming tasks, with recent studies indicating that students often find code explanations generated by LLMs to be more comprehensible than those provided by peers [2]. Therefore, it is no surprise that universities begin to integrate LLM-based code-feedback systems such as [3]–[5] into their courses.

While we do not subscribe to the notion of an imminent “end of programming” [6], we recognize that GenAI has already reshaped the learning practices of novice program-

mers [7]. Consequently, the computing education community must respond to these shifts [8] by making research-informed decisions about how to operate in this new environment. It is therefore imperative to understand how GenAI influences learning outcomes—both positively and negatively—and the extent to which these tools reshape the student learning experience.

Though evidence suggests that LLMs enhance productivity in programming tasks [9]–[11], their impact on learning outcomes remains unclear. Studies present mixed findings: some report improved learning outcomes with LLM integration in programming courses [12], while others find no significant effect [13] or even negative outcomes [14]. Moreover, many of these studies rely on students’ self-reported perceptions, which may not accurately reflect actual learning gains [15].

A further concern is the potential for language models to amplify skill disparities. Evidence indicates that programming experience is crucial not only for effectively utilizing large language models (LLMs) in task-solving but also for enhancing one’s programming skills [16]. While competent students may use generative AI (GenAI) effectively as a supportive tool, students with metacognitive challenges may struggle or bypass the engagement with challenging problems by simply generating solutions [15]. Since tools like ChatGPT [17] and GitHub Copilot [18] were developed primarily for the industry rather than education, they inadvertently solve programming problems for novice students—a major concern regarding the use of LLMs in educational settings [8], [19]—potentially depriving them of critical learning experiences.

Our study addresses this research gap by investigating student interactions with the CS50 Duck, an AI-powered chatbot designed as a virtual “rubber duck” for Harvard’s CS50 course [5]. We adapted the CS50 Duck for our Computational Thinking course, which shares key characteristics with Harvard’s CS50. Notably, students accessed the chatbot solely through Harvard’s web interface (<https://cs50.ai/chat>).

Harvard’s tool might encourage responsible GenAI use by offering students an experience contrasting ChatGPT’s types of responses with a Socratic dialogue that emphasizes the cognitive engagement required for meaningful learning.

Our study examines how students perceive and interact with the CS50 Duck relative to other GenAI tools, exploring whether such language models might help us approach an educational ideal: achieving a 1:1 teacher-to-student ratio. To this end, we address the following research questions:

- RQ1** How do students perceive the effectiveness of the CS50 Duck?
- RQ2** Do students exhibit subjective reliance on the CS50 Duck or other GenAI tools?
- RQ3** How do students use the CS50 Duck and other GenAI tools?
- RQ4** What are students' expectations for using GenAI tools in programming courses?

II. METHODOLOGY

In this study, we used a mixed-methods approach involving a questionnaire and a practical assignment to gain a comprehensive understanding of students' experiences with GenAI tools in programming education. The questionnaire, which students completed just over one month into the semester, captured students' self-reported attitudes, prior knowledge, and usage patterns, while the assignment, completed one week later, provided a hands-on opportunity to observe the students' application of the CS50 Duck in a structured problem-solving scenario. This two-part methodology allows us to assess both subjective perceptions and practical engagement with GenAI tools. Note that instructors motivated the use of the CS50 Duck by introducing it to the students.

A. Survey Design and Distribution

The survey comprised a structured five-page digital questionnaire with a total of 30 items, listed in Table I & II, adapted with reference to prior survey instruments [4], [20], [21]. The questionnaire included 28 closed-ended items, primarily multiple-choice, and two open-ended items (Q29-Q30). Six questions (Q1-Q2 & Q20-Q23) required categorical selections, one question prompted item ranking, and the remainder utilized a five-point Likert scale ranging from "Strongly disagree" to "Strongly agree" for assessing agreement with specific statements.

The questionnaire was generated using *SoSci Survey* [22] and was made available to participants via www.soscsurvey.de. The platform offers a solution, compliant with German data protection and accessibility regulations. For scientific surveys that are not conducted on behalf of a partnership or a company, *SoSci* is free of charge.

Students were given 20 minutes during the lecture to voluntarily complete the anonymous survey. The topic was introduced in advance to engage students, and the objectives of the survey were communicated transparently. Additionally, it was explained that the ultimate aim is to further integrate GenAI tools and establish them as a core component of the teaching and learning environment.

The introductory page provided participants with an overview of the survey's parameters, including a hint to be able to withdraw at any time. The second page collected

demographic and contextual information, querying students on their academic term (Q2), field of study (Q1), and prior knowledge (Q3-Q4).

Given that terms like "GenAI tools" and "CS50 Duck" are central to the questionnaire, we included an additional page to define these concepts and confirm that students understood the terminology used throughout (Q5-Q6).

Pages 4 and 5 focused on students' usage patterns and perceptions regarding the CS50 Duck and GenAI tools in the context of programming education. Questions Q7-Q19 specifically investigated how students employ the CS50 Duck in their learning process (e.g., Q7, Q8) and their perceptions of its effectiveness in enhancing problem-solving skills (Q10), comprehension (Q11), and motivation (Q13). Additional questions assessed students' comfort levels in interacting with the CS50 Duck (Q14, Q15) and compared preferences for its use over human tutors (Q16, Q17). Certain items further explored students' reliance on the CS50 Duck for programming tasks and exams, evaluating its perceived necessity for successful assignment completion (Q18, Q19).

Questions Q24-Q28, listed in Table II, addressed anticipated future usage of GenAI tools in programming education (Q24), probing students' beliefs regarding the potential for frequent AI use to impact learning outcomes (Q25). They also examined students' perceptions of AI effectiveness relative to human instructors (Q26), the likelihood of AI replacing human instructors (Q27), and the importance of GenAI proficiency for career readiness (Q28).

Finally, the open-ended questions Q29 and Q30 invited students to provide reflective insights and describe specific usage practices related to GenAI tools, such as the CS50 Duck, within the course.

B. Assignment Design and Distribution

One week after finishing the questionnaire students received a specific assignment distributed via the Moodle platform. They were asked to voluntarily solve the assignment using the CS50 Duck. The problem was to compute the column and row sum of a matrix represented by a Python list of lists. To this point, Python lists were already introduced in the lecture but lists of lists were not. The assignment reads the following:

Assignment

A company wants to obtain a detailed overview of the sales figures of its three branches (A, B, and C) for an entire year. The monthly sales figures are already stored in a nested list (a list with lists as elements). Each sublist contains the revenues of a branch for each month.

- (1) Calculate the total annual revenue for each branch.
- (2) Calculate the total monthly revenue of all branches combined.
- (3) Determine the branch with the highest total annual revenue and output its name (branch A, B, or C).

Example output (for the example values). The output could look as follows:

```
Total revenue of branch A: 30000
Total revenue of branch B: 40000
Total revenue of branch C: 35000
```

```
Total revenue in January: 8250
Total revenue in February: 8250
...
Total revenue in December: 10050
```

```
Branch with the highest annual revenue: Branch B
```

TABLE I: First 19 questioner questions assessing students’ background, confidence in programming, usage patterns, perceptions, and attitudes toward the CS50 Duck and other GenAI tools.

Abbr.	Question
Q1	What program are you studying?
Q2	What semester are you in?
Q3	I feel confident in using the computer.
Q4	Please rate your programming skills at the beginning of the semester on a scale from 1 (nonexistent) to 5 (excellent).
Q5	I know what is meant by “GenAI tools”.
Q6	I know what is meant by “CS50 Duck”.
Q7	I regularly use the CS50 Duck when working with code.
Q8	I regularly use other GenAI tools besides the CS50 Duck (e.g., ChatGPT) when working with code.
Q9	The CS50 Duck “understands” my queries.
Q10	The CS50 Duck provides support that directly helps me with problems I encounter when working on a programming task.
Q11	The guidance provided by the CS50 Duck has improved my understanding of programming concepts.
Q12	Interacting with the CS50 Duck makes the learning process more engaging.
Q13	I feel more motivated to work on programming tasks when I use the CS50 Duck.
Q14	I feel comfortable asking the CS50 Duck questions without worrying about being judged.
Q15	I feel safe asking the CS50 Duck questions that I would not feel comfortable asking tutors or professors.
Q16	I prefer to ask the CS50 Duck questions instead of asking a human tutor for help.
Q17	I would prefer to ask the CS50 Duck questions about lecture material rather than asking the professor.
Q18	Without the CS50 Duck (and other chatbots like ChatGPT), I would find it more difficult to solve programming tasks.
Q19	I imagine it would be challenging to complete tasks in computer-based exams without using the CS50 Duck.

Ask the CS50 Duck! Use the Duck as a tutor/sparring partner to gain knowledge on topics such as programming, lists, loops, or simply how to start structuring the problem.

```
# Given list of sales figures for each month and each branch
revenue = [
    [2100, 3400, 2750], # January: Revenue of branches A, B, C
    [2150, 3200, 2900], # February: Revenue of branches A, B, C
    [2200, 3150, 3000], # March: Revenue of branches A, B, C
    [2300, 3100, 2800], # April: Revenue of branches A, B, C
    [2400, 3250, 2950], # May: Revenue of branches A, B, C
    [2500, 3300, 3100], # June: Revenue of branches A, B, C
    [2550, 3400, 3200], # July: Revenue of branches A, B, C
    [2600, 3450, 3300], # August: Revenue of branches A, B, C
    [2650, 3500, 3400], # September: Revenue of branches A, B, C
    [2700, 3550, 3450], # October: Revenue of branches A, B, C
    [2750, 3600, 3500], # November: Revenue of branches A, B, C
    [2800, 3650, 3600], # December: Revenue of branches A, B, C
]
```

III. RESULTS

This section presents the outcomes of the questionnaire responses and assignment observations, capturing both students’ general impressions and specific usage patterns of GenAI tools, particularly the CS50 Duck.

A. General Impressions

The questionnaire received a total of 72 valid responses from BSc students enrolled in the Computational Thinking course. Among the respondents, 10 study Geodata Science, 27 are in Digital Engineering, and 35 are in Computer Science and Design. Only one participant was from the third semester, with the remainder in their first. Figure 1 displays the distribution of responses for questions Q3-Q4 and Q7-Q19 on a five-point Likert scale.

The majority of respondents reported feeling comfortable working with computers (Q3) and demonstrated an understanding of terms such as “GenAI tools” and “CS50 Duck”. However, self-assessed programming competence varied (Q4),

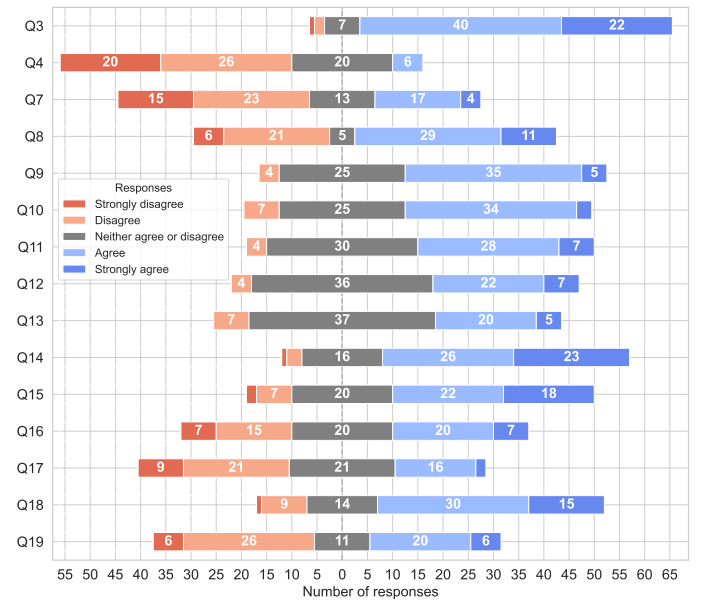


Fig. 1: Stacked bar chart showing the distribution of responses for questions Q3-Q4 and Q7-Q19 on a five-point Likert scale.

as expected, given that many students entering universities of applied sciences in Germany come from apprenticeship backgrounds.

No significant differences were observed between students across the different academic disciplines.

TABLE II: Second part of the questionnaire assessing students' background, confidence in programming, usage patterns, perceptions, and attitudes toward the CS50 Duck and other GenAI tools.

Abbr.	Question
Q20	Which of the following statements best describes how GenAI tools (e.g., ChatGPT, CS50 Duck, GitHub Copilot) generate answers in text form? 1) They understand language like humans and try to respond based on what they understand. 2) They analyze patterns in large datasets and generate responses based on statistical probabilities. 3) They are designed to empathize with people and generate responses based on shared experiences. 4) They are based on a rule-based system of fixed responses pre-programmed by developers.
Q21	If you had a question about the course material or encountered a problem, in what order did you take the following steps? Please rank the options below. 1) Ask CS50 Duck 2) Ask a GenAI tool different from the CS50 Duck (e. g. ChatGPT) 3) Ask in the Moodle forum 4) Search online (e. g. Google, StackOverflow) 5) Ask a classmate 6) Ask the teaching assistants/professor
Q22	After generating code with GenAI tools for my tasks in this course, I mainly did the following: 1) Not applicable (I did not use any GenAI tools, or no code was generated) 2) I used the code directly. 3) I briefly reviewed the code to ensure it looked correct. 4) I carefully read the code to ensure it was correct. 5) I carefully read the code and wrote test code to ensure it was correct. 6) I submitted the newly generated code to the GenAI tool and had it checked for errors.
Q23	For programming assignments of this course, I believe GenAI tools should be used as follows: 1) Always allow all GenAI tools (e.g., including ChatGPT). 2) Always allow the CS50 Duck, prohibit everything else. 3) Always prohibit all GenAI tools (including the CS50 Duck). 4) Allow all GenAI tools for some assignments and prohibit all for others (depending on task type, difficulty, etc.). 5) Allow only the CS50 Duck for some assignments and prohibit all GenAI tools for others.
Q24	I expect to increasingly use GenAI tools in my learning practices for programming in the future.
Q25	The frequent use of GenAI tools for code generation is harmful to my learning of programming.
Q26	GenAI tools for programming can provide guidance in programming tasks as effectively as human instructors.
Q27	GenAI tools will replace human instructors in the future.
Q28	Students need to learn how to use GenAI tools effectively in order to be prepared for their future careers.
Q29	Can you explain in more detail when you think the use of GenAI tools in this course should be allowed or prohibited?
Q30	Describe how and which GenAI tools you used in this course (e.g., code generation, code explanation, summarizing tasks, etc.).

B. Students' Perception of GenAI

58% of students reported that the CS50 Duck effectively comprehends their inquiries, while 5.8% disagreed (Q9). For direct assistance, 54% agreed that the CS50 Duck was helpful, 36% were neutral, and 10% disagreed (Q10). Regarding its role in enhancing programming comprehension (Q11), 51% of students responded positively, 43% were neutral, and 6% disagreed.

Responses to Q9–Q11 were strongly positively correlated, indicating consistency in perceptions of the CS50 Duck's effectiveness across these dimensions, compare Figure 2. Additionally, responses to Q7 showed a positive correlation with those of Q9–Q13, suggesting that frequency of use correlates with perceived usefulness. Interestingly, the use of other GenAI tools correlated negatively with the perceived usefulness of the CS50 Duck.

Of the respondents, 58 (80.5%) selected the description of GenAI that we consider the most accurate (Q20-2), while 9 (12.5%) appeared to believe that these tools understand language in the same way humans do, compare Figure 3.

Responses regarding engagement with the CS50 Duck were largely neutral. While 42% of students found interactions

engaging, 52% were neutral, and 6% disagreed (Q12). Motivation levels were similarly moderate (Q13). A substantial 71% of students felt comfortable asking questions to the CS50 Duck without fear of judgment (Q14), and 58% felt safe asking questions generally, though 13% disagreed (Q15). Therefore, we reproduced the findings of Liu et al. [5].

Responses to Q9-Q14 were positively correlated, with a particularly strong correlation observed between Q12 and Q13, indicating a generally positive attitude towards the CS50 Duck. Interestingly, programming proficiency (Q4) showed a negative correlation with both the fear of judgment (Q14) and a perceived reliance on the CS50 Duck during computer-based exams (Q19). This relationship may have multiple contributing factors; for instance, proficient programmers are often older. As shown in Figure 2, Q13 also correlates with Q19, but this correlation is not statistically significant, as the p -value exceeds 0.05.

When students were asked about their preference for consulting the CS50 Duck over a human teaching assistance (Q16), responses were nearly split. However, for questions related to lecture content (Q17), the human teaching assistance was preferred.

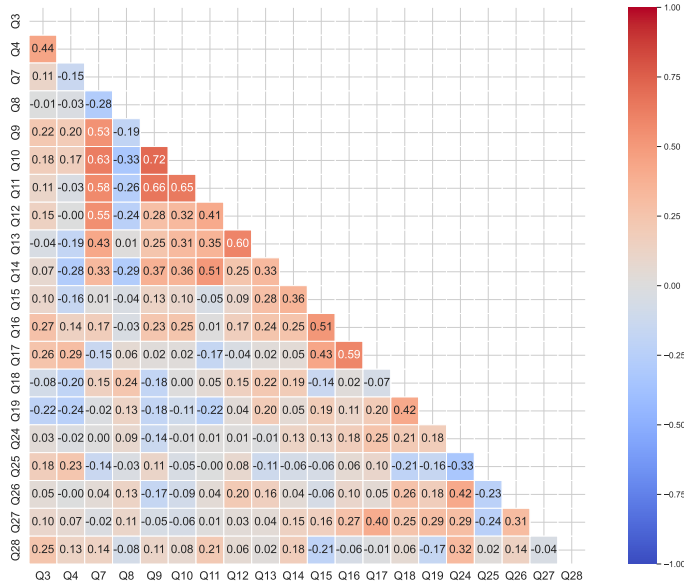


Fig. 2: Heatmap of Spearman's rank correlation coefficient matrix for the items of Q3-Q28. Positive and negative correlations between variables are depicted using a diverging color scale, where values range from -1 (strong negative correlation) to 1 (strong positive correlation).

A majority of students (65%) indicated that completing programming tasks would be more challenging without GenAI tools (Q18). This opinion shifts in the context of computer-based exams, where students feel less reliant on the CS50 Duck specifically (Q19). This suggests that while GenAI tools are generally perceived as beneficial (more beneficial than the CS50 Duck) for task completion, their utility may be seen as situational.

68% of students anticipate an increase in GenAI usage in their programming learning practices in the future (Q24), while only 10% disagree. Regarding whether frequent code generation may be harmful to learning programming (Q25), student opinions are nearly evenly divided. We found no correlation between students' reported use of GenAI (Q9 & Q10) and their opinions on the harmfulness of code generation (Q25). However, students who anticipate increased use in the

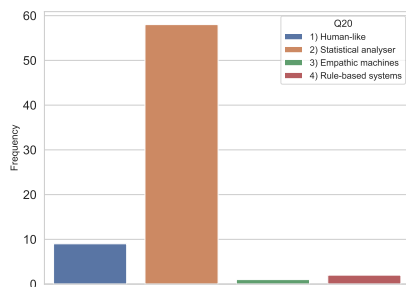


Fig. 3: Students' choices for the best description of GenAI tools (Q20).

future (Q24) tend to perceive its usage as less harmful (Q25). Figure 2 shows this weak negative correlation.

The majority of students 37 (52%) believe that GenAI tools can provide guidance in programming tasks as effectively as human instructor (Q26) and 32% disagree with the statement. Nonetheless, a significant majority (73%) does not believe that GenAI will fully replace human instructors, though 14% think it is possible.

In relation to career preparedness (Q28), 87% of students believe they should be equipped to use GenAI tools effectively, with only one student disagreeing with this statement.

C. Students' Usage of GenAI

Overall, students utilized the CS50 Duck more frequently than professors and teaching assistants anticipated. Observations indicated that students engaged with the tool in a dialogic manner, using it as a virtual tutor. But students reported using other GenAI tools more frequently than the CS50 Duck when working with code (Q7–Q8), which is also consistent with the instructors' observation. Despite the university providing free access to ChatGPT-3.5 Turbo and ChatGPT-4, most students continue to use their personal accounts. Those with prior programming knowledge tend to use the CS50 Duck less often; however, no correlation was found between prior experience and the use of other GenAI tools. A majority of students have always at least one GenAI tool open while working in class. Given that 15 students (20%) strongly disagree with statement Q7, it can be inferred that these students make minimal use of the CS50 Duck. Interestingly, students identified by instructors as significantly struggling appeared to avoid using GenAI tools in class entirely.

Notably, no student reported using AI-generated code directly without modification (Q22). However, instructors observed that students tend to turn to GenAI tools (particularly ChatGPT) when they are directly seeking solutions or when they lose patience with the CS50 Duck. A majority of 27 (49%) students reported carefully reviewing and testing the generated code before using it. 18 (33%) students reported to read the code thoroughly without additional testing, 6 (11%) only skim through it, and 4 (7%) use GenAI tools to verify code correctness. 16 students indicated that this question was not applicable, likely because the CS50 Duck primarily generates short code snippets rather than complete code solutions. We excluded them from the computation of the percentages.

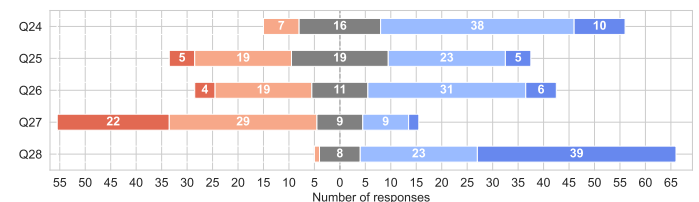


Fig. 4: Stacked bar chart showing the distribution of responses for questions Q24-Q28 on a five-point Likert scale.

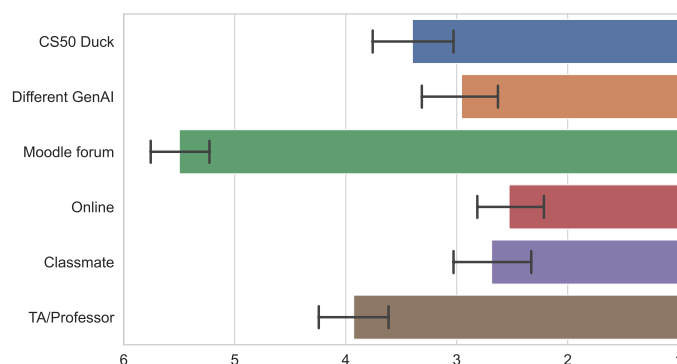


Fig. 5: Average order in which students seek help (Q21). Low numbers indicate a high priority. This order was reported by students. The bars mark the 95% confidence intervals. Since they overlap, some of the differences between categories are statistically insignificant.

Interestingly, when students encounter difficulties during problem-solving (Q21), they prioritize online searches and consulting classmates over using GenAI tools, while teaching assistants and professors are ranked even lower, compare Figure 5. Instructors also observed that students tend to consult their classmates before turning to a GenAI tool for assistance. It is important to note that students were asked to provide an order of preferred help-seeking methods, which does not necessarily reflect their overall valuation of each resource. This ordering suggests a reasonable and desirable approach, with students initially attempting to resolve issues independently before seeking assistance from teaching assistants or professors. The preference of online search and asking a classmate over GenAI tools requires a different explanation, such as familiarity or effectiveness but this difference is also statistically insignificant.

D. Students' Attitudes Toward GenAI

To keep the questionnaire focused and concise, we did not delve into ethical considerations, though we recognize their importance. However, we asked students for their opinions on the appropriate level of GenAI tool usage in course programming assignments (Q23). The largest group (31%) favored unrestricted use of all GenAI tools (Q23-1), followed by 29% who supported allowing all GenAI tools for specific assignments only (Q23-4). Meanwhile, 27% preferred a restricted allowance limited to the CS50 Duck (Q23-5), 10% opted for unrestricted use of the CS50 Duck alone (Q23-2), and 3% advocated for banning all GenAI tools in these assignments (Q23-3). Thus, a majority of 69% favors some form of restriction, a sentiment also reflected in the responses to Q29.

Students' opinions on the use of GenAI tools in the course (Q29) reveal diverse but generally favorable attitudes toward their inclusion as learning aids, with some nuanced preferences regarding their applications. A significant number of students advocate for unrestricted access to GenAI tools like the CS50

Duck, especially for practice and self-guided learning, as they believe such tools enhance understanding, reduce the time spent on troubleshooting, and support independent problem-solving. Many students emphasize that GenAI should primarily assist in finding initial directions, clarifying code syntax, or offering explanations rather than providing full solutions, as they recognize the potential for over-reliance to hinder deep learning.

"For complex tasks that require programming skills, AI tools can certainly be helpful. However, constantly relying on these tools can hinder personal development, as they are not always effective and can sometimes produce incorrect or overly verbose code that may be confusing."

Some students also draw parallels between GenAI tools and other standard learning aids, like calculators, noting the importance of learning how to use these tools responsibly and effectively, as they are likely to be integral in their future careers.

Conversely, another group of students express concerns about GenAI tools in contexts where independent skill assessment is critical, such as exams or assignments intended to evaluate core programming competencies. There is a general consensus that, while helpful, these tools should be restricted in examinations to ensure individual understanding is accurately measured.

"For exercises and assignments that are not graded, I believe using such tools is acceptable. However, in exams, the use of these tools is not appropriate."

Overall 54 (75%) gave their opinion on this question. Students recognize both the benefits of GenAI tools in accelerating and enriching the learning process and the need for thoughtful boundaries to maintain educational integrity and skill development.

The students' responses regarding question Q30 indicate that they primarily use GenAI tools, such as ChatGPT, the CS50 Duck, and GitHub Copilot, for a variety of supportive functions in programming tasks. Key applications include code explanation, troubleshooting errors, and syntax clarification, particularly when they encounter challenging concepts or are unsure about specific implementations. Many students reported to use GenAI tools as a first step to gain understanding, viewing these tools as a supplement to self-study and traditional resources rather than a replacement. For example, ChatGPT and the CS50 Duck are frequently used for clarifying assignment instructions, verifying code logic, and exploring alternative solutions.

"ChatGPT: When I didn't understand a topic (often while working on a lab assignment), I would ask a general question about the topic without copying anything directly from the assignment. I would read through the general explanation and then try to solve the assignment myself. If that didn't help, my next

step was to paste the assignment into ChatGPT but only request hints, explicitly stating that I wanted to maintain a learning effect. In an absolute last resort, I would have the solution generated and then critically review and question it.”

There is also a trend toward using GenAI tools to identify and correct mistakes, especially in assignments where students feel uncertain about their solutions. Some students approach the tools strategically, initially asking for hints rather than direct answers to maintain a learning effect. Notably, a few students mentioned the convenience of these tools in adapting to different programming languages and bridging knowledge gaps independently. However, some students also use these tools for code generation.

“ChatGPT and CS50 Duck: Code explanation and code generation.”

Overall, students express appreciation for GenAI as a flexible and accessible aid that complements their coursework by enhancing comprehension and problem-solving efficiency.

E. Students’ Interaction Patterns

In our study, 8 students submitted their chat history illustrating their interactions with the CS50 Duck while working on the assignment introduced in Section II-B. Solely based on their chat history and our experience as educators, we classified students into two groups: *proficient* (P1–P5) and *struggling* (P6–P8).

Proficient students effectively leveraged the CS50 Duck to troubleshoot and progress through the assignment. These students posed precise questions, demonstrating awareness of the information previously given to the CS50 Duck. Notably, only in the case of P4 did the CS50 Duck generate code in response to specific queries. Typically, students in this group shared their code snippets, which the CS50 Duck then analyzed and provided suggestions for subsequent steps. Below is a typical excerpt from one such dialogue:

Chat history

...
CS50 Duck: It looks like you have a list of monthly turnovers for different branches. How can I assist you with this data? Do you have a specific question or task you need help with?

P1: If I have a list like:

```
for i in turnover:
    for j in i:
```

can I sum my j's up ?

CS50 Duck: Yes, you can sum up the values of j within the nested loops. How do you think you could keep track of the sum as you iterate through the list?

P1: I use a variable like `sumup_monthly` and set it to 0.

CS50 Duck: Yes, initializing a variable like `sumup_monthly` to 0 is a good start. How would you update this variable within your nested loops to keep a running total?

P1: `sumup_monthly += j`
...

Struggling students demonstrated difficulties in communication, often providing inaccurate information, posing ambiguous or misleading questions, and misinterpreting the Duck’s knowledge scope. They also produced more code, presumably unintentionally—that is, without directly requesting it. For instance, P6 and P7 asked for assistance in summing up all matrix values instead of requesting monthly or branch-specific totals. Additionally, P8 presented their task in an unclear manner:

“The second task is to sum the monthly sales of the three shops together.”

P7 initiated with a request for a “101 of nested lists” which the CS50 Duck supplied. Subsequently, the dialogue continued as follows:

Chat history

P7: Hi, I need to solve a programming task with nested lists. Could you give me a quick 101 summary on nested lists?

CS50 Duck: [Explanation of nested lists]

P7:

```
def revenue_year(filliale: str) -> int:
    counter = 0
    if branch == "A":
        for i in revenue[counter][0]:
            revenue_branch += i
            counter += 1
    print(revenue_branch)
    revenue_year("A")
```

Where is my thinking wrong?

CS50 Duck: [Hints at (1) the missing initialization of `revenue_branch`, (2) the problematic increment of `counter` and (3) asks for information about `revenue`.]

P7: [Provides the list `revenue` to the CS50 Duck by copying it from the assignment description including the comments.]

CS50 Duck: Thank you for the additional information! It looks like you want to calculate the revenue for branch A. Here are a few adjustments you could make: (1) Initialization of `revenue_branch`: Make sure that `revenue_branch` is initialized. (2) Loop index: Ensure that you are iterating over the correct column for branch A. Here is an approach you could try:

```
def revenue_year(branch: str) -> int:
    revenue_branch = 0
    if branch == "A":
        for month in revenue:
            revenue_branch += month[0]
    return revenue_branch
    revenue_year("A")
```

Can you try this approach and see if it works?

...

In this instance, the CS50 Duck generated correct code based on contextual information available in the assignment and included comments. This type of interaction continued with the CS50 Duck correcting semantically incorrect code provided by the student, raising a potential issue: the CS50 Duck may offer overly detailed solutions, especially if the student is struggling.

As anticipated, the CS50 Duck can detect errors in student code when sufficient context is given. However, it does not necessarily discern the underlying intent behind a student’s code. For example:

Chat history

...
P6: The third task is to name out the shop with the most sales and its yearly sales.

CS50 Duck: To determine which shop has the most sales and its yearly sales, you can follow these steps: (1) Calculate the total yearly sales for each shop. (2) Compare the totals to find the shop with the highest sales. Can you try implementing this and show me what you come up with?

P6:

```
def best_shop_and_its_sales():  
    ...  
    if year_sales[0] > year_sales[1] and year_sales[2]:  
        return year_sales[0], first_shop  
    elif year_sales[1] > year_sales[0] and year_sales[2]:  
        return year_sales[1], second_shop  
    else:  
        return year_sales[0], third_shop
```

CS50 Duck: It looks like you're on the right track! However, there are a couple of issues in your comparison logic. Specifically, the conditions in your `if` and `elif` statements need to be adjusted. Consider this: You should compare each shop's sales individually. Ensure that the correct shop is returned based on the highest sales. Here's a hint to help you correct the logic:

```
if year_sales[0] > year_sales[1] and year_sales[0] > year_sales[2]:  
    return year_sales[0], first_shop  
elif year_sales[1] > year_sales[0] and year_sales[1] > year_sales[2]:  
    return year_sales[1], second_shop  
else:  
    return year_sales[2], third_shop
```

Try updating your function with this logic and see if it works!

...

A human instructor—who experienced all sorts of errors and is provided with a “feeling” for his group—would likely recognize that the student’s intent might be, in fact, correct. At this point the instructor could offer a more nuanced feedback, while the Duck’s responses primarily focused on structural correction rather than intent. This tendency to provide exact answers (and silently accept suboptimal solutions) may also inadvertently discourage independent problem-solving, as demonstrated in P7’s dialogue.

In some cases, the questions and responses from the CS50 Duck seemed to sidetrack the student, as observed with P8. Attempting to understand the `.join()` function for summing numbers of a list, P8 then requested an explanation of `map()` and, after multiple interactions, produced incorrect code. The Duck’s responses, though technically accurate, did not prevent further missteps by P8.

Chat history

...
CS50 Duck: The issue is that `full_sales` is a string, and you are trying to call it as if it were a function. Instead, you should be iterating over the original revenue list directly. Here's a hint on how to approach it:

- Iterate over the revenue list.
- Access the correct index for each sublist based on the value of `a`.

For example, if `a == 1`, you want to sum the first element of each sublist. Here's a conceptual approach:

```
if a == 1:  
    for inner_list in revenue:  
        count += inner_list[0]  
print(f"Overall revenue of branch A: {count}")
```

Try applying this logic to your function for each case (`a == 1`, `a == 2`, `a == 3`).

P8: How to convert tuples into integers or slices?

...

Even when a question is only marginally coherent, the CS50 Duck still attempts to provide positive feedback, contributing to a perception of “inhuman patience”. This tendency has also

been observed by [23] (for ChatGPT) and [5] (for the CS50 Duck). However, it remains unclear whether a more judicious response to misunderstandings in student questions would be advantageous.

IV. CONCLUSION

By investigating students’ interactions with the CS50 Duck and other generative AI tools, we observed patterns in usage, perceptions, and the potential influence of these tools on learning practices and outcomes. The analysis of the chat history indicates that while proficient students leverage these tools as effective aids to deepen understanding and problem-solving skills, struggling students are more likely to get distracted and may rely on them in ways that could hinder self-directed learning.

In the right hands, the CS50 Duck is a powerful tool that accelerates the programming learning process, making it more enjoyable, safe, and, importantly, less intimidating. It encourages novices to ask questions freely. However, when used as a shortcut, it can produce more of the solution than instructors might intend. And even without generating code, it may take over the critical task of (simulating) computational thinking. Therefore, simply incorporating it into traditional lessons without further adjustments is not sufficient.

Student feedback indicates that LLMs like the CS50 Duck offer significant benefits, particularly by providing consistent, non-judgmental support that encourages frequent engagement with programming challenges echoing the result of Liu et al. [5]. However, this reliance raises questions about the balance between AI-assisted guidance and the development of independent problem-solving abilities. The potential for over-reliance among less proficient students points to the need for instructional strategies that emphasize responsible and strategic use of GenAI tools.

While students generally view LLMs positively and provide a solid description of them, our analysis suggests that their effectiveness varies depending on students’ skill levels and their ability to contextualize feedback from AI. As a result, there is an essential role for educators in framing these tools as supplements rather than replacements for foundational learning processes. In contexts where LLMs could substitute direct problem-solving efforts, such as graded assignments or exams, there may be a need for controlled usage to ensure that learning outcomes accurately reflect students’ abilities.

Our findings contribute to the growing discourse on integrating generative AI in education by highlighting its benefits and limitations as a scalable support tool in programming courses. As AI-powered tools become more sophisticated and widely accessible, future research should explore how their design and deployment can better align with educational goals, addressing skill disparities and promoting an adaptive learning experience tailored to individual needs.

V. LIMITATIONS

While our study provides valuable insights into student interactions with the CS50 Duck and other generative AI

tools, several limitations should be acknowledged. First, our sample size was relatively small and limited to students within a specific Computational Thinking course. This may affect the generalizability of our findings to other programming courses or to students with differing levels of experience in programming.

Additionally, our reliance on self-reported data through questionnaires, while useful for capturing students' perceptions and attitudes, may not fully represent their actual engagement with GenAI tools, a problem discovered by, e.g. [15] and [24]. The difference we observed in chat histories and the students' statements hints to this problem.

Moreover, while we analyzed student-chatbot interactions in an assignment context, the lack of real-time observation may have limited our understanding of how students use the CS50 Duck in less structured or unsupervised environments.

Finally, although we observed that certain students used GenAI tools in ways that might undermine independent problem-solving, our study does not provide a longitudinal perspective on how these tools impact learning outcomes over time.

Future research could address these limitations by including a larger and more diverse sample, employing observational data collection methods, and assessing the long-term effects of GenAI tool usage on programming proficiency and computational thinking skills.

REFERENCES

- [1] J. Prather, P. Denny, J. Leinonen, B. A. Becker, I. Albluwi, M. E. Caspersen, M. Craig, H. Keuning, N. Kiesler, T. Kohn, A. Luxton-Reilly, S. MacNeil, A. Petersen, R. Pettit, B. N. Reeves, and J. Savelka, "Transformed by transformers: Navigating the ai coding revolution for computing education: An iticse working group conducted by humans," in *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 2*, ser. ITiCSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 561–562. [Online]. Available: <https://doi.org/10.1145/3587103.3594206>
- [2] J. Leinonen, P. Denny, S. MacNeil, S. Sarsa, S. Bernstein, J. Kim, A. Tran, and A. Hellas, "Comparing code explanations created by students and large language models," in *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, ser. ITiCSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 124–130. [Online]. Available: <https://doi.org/10.1145/3587102.3588785>
- [3] S. Jacobs and S. Jaschke, "Evaluating the application of large language models to generate feedback in programming education," in *2024 IEEE Global Engineering Education Conference (EDUCON)*, 2024, pp. 1–5.
- [4] P. Bassner, E. Frankford, and S. Krusche, "Iris: An AI-driven virtual tutor for computer science education," in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, ser. ITiCSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, pp. 394–400. [Online]. Available: <https://doi.org/10.1145/3649217.3653543>
- [5] R. Liu, C. Zenke, C. Liu, A. Holmes, P. Thornton, and D. J. Malan, "Teaching CS50 with AI: Leveraging generative artificial intelligence in computer science education," in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. ACM, 2024, pp. 750–756. [Online]. Available: <https://dl.acm.org/doi/10.1145/3626252.3630938>
- [6] M. Welsh, "The end of programming," *Communications of the ACM*, vol. 66, no. 1, pp. 34–35, 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3570220>
- [7] S. Hammer, S. Ottinger, B. Zönnchen, M. Hohendanner, M. Hobelsberger, and V. Thurner, "ChatGPT in higher education: Perceptions of computer science-related students," in *2024 IEEE Global Engineering Education Conference (EDUCON)*, 2024, pp. 01–08.
- [8] B. A. Becker, P. Denny, J. Finnie-Ansley, A. Luxton-Reilly, J. Prather, and E. A. Santos, "Programming is hard – or at least it used to be: Educational opportunities and challenges of AI code generation," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, ser. SIGCSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 500–506. [Online]. Available: <https://doi.org/10.1145/3545945.3569759>
- [9] A. Carleton, D. Falessi, H. Zhang, and X. Xia, "Generative AI: Redefining the future of software engineering," *IEEE Software*, vol. 41, no. 6, pp. 34–37, 2024.
- [10] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, "Measuring GitHub Copilot's impact on productivity," *Commun. ACM*, vol. 67, no. 3, p. 54–63, feb 2024. [Online]. Available: <https://doi.org/10.1145/3633453>
- [11] J. Israilidis, W.-Y. Chen, and M. Tsakalerou, "Software development and education: Transitioning towards AI enhanced teaching," in *2024 IEEE Global Engineering Education Conference (EDUCON)*, 2024, pp. 1–6.
- [12] A. Vadaparty, D. Zingaro, D. H. Smith Iv, M. Padala, C. Alvarado, J. Gorson Benario, and L. Porter, "CS1-LLM: Integrating LLMs into CS1 instruction," in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. ACM, 2024, pp. 297–303. [Online]. Available: <https://dl.acm.org/doi/10.1145/3649217.3653584>
- [13] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of AI code generators on supporting novice learners in introductory programming," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ser. CHI '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3544548.3580919>
- [14] G. Jošt, V. Taneski, and S. Karakatič, "The impact of large language models on programming education and student learning outcomes."
- [15] J. Prather, B. Reeves, J. Leinonen, S. MacNeil, A. S. Randrianasolo, B. Becker, B. Kimmel, J. Wright, and B. Briggs, "The widening gap: The benefits and harms of generative AI for novice programmers," 2024. [Online]. Available: <https://arxiv.org/abs/2405.17739>
- [16] B. Zönnchen, V. Thurner, and A. Böttcher, "On the impact of ChatGPT on teaching and studying software engineering," in *2024 IEEE Global Engineering Education Conference (EDUCON)*, 2024, pp. 1–10.
- [17] OpenAI, "GPT-4 technical report," 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [18] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [19] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günemann, E. Hüllermeier, S. Krusche, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Sailer, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kuhn, and G. Kasneci, "ChatGPT for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1041608023000195>
- [20] H. Keuning, I. Alpizar-Chacon, I. Lykourantzou, L. Beehler, C. Köppe, I. de Jong, and S. Sosnovsky, "Students' perceptions and use of generative AI tools for programming across different computing courses," 2024. [Online]. Available: <https://arxiv.org/abs/2410.06865>
- [21] J. Prather, P. Denny, J. Leinonen, B. A. Becker, I. Albluwi, M. Craig, H. Keuning, N. Kiesler, T. Kohn, A. Luxton-Reilly, S. MacNeil, A. Petersen, R. Pettit, B. N. Reeves, and J. Savelka, "The robots are here: Navigating the generative AI revolution in computing education," in *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, ser. ITiCSE-WGR '23.

- New York, NY, USA: Association for Computing Machinery, 2023, pp. 108–159. [Online]. Available: <https://doi.org/10.1145/3623762.3633499>
- [22] D. J. Leiner, “SoSci Survey (version 3.5.01).” [Online]. Available: <https://www.soscisurvey.de>
- [23] A. Hellas, J. Leinonen, S. Sarsa, C. Koutchene, L. Kujanpää, and J. Sorva, “Exploring the responses of large language models to beginner programmers’ help requests,” in *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1*, ser. ICER ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 93–105. [Online]. Available: <https://doi.org/10.1145/3568813.3600139>
- [24] D. L. McCabe, L. K. Trevino, and K. D. Butterfield, “Cheating in academic institutions: A decade of research,” *Ethics & Behavior*, vol. 11, no. 3, pp. 219–232, 2001. [Online]. Available: https://doi.org/10.1207/S15327019EB1103_2